



INTERNAL — DEV TEAM BRIEF

PDF Takeoff Tool

Proof of Concept Brief

CLIENT Nathan(Sydney, NSW)	DEMO AUDIENCE Nathan + Andrew(technical lead)	OWNER Lavesh	DEMO VIA Screenshare
--	---	------------------------	--------------------------------

SECTION 1

Why This POC Exists

Sales has committed to Nathan a working proof of concept as the entry point to the full build engagement. The POC is a contractual hook for Milestone 1 sign-off, which unlocks the broader project.

This is not a research project. It is a demo. The goal is to show Nathan that our extraction approach works on his real data, end to end, before he commits to the build. Anything that doesn't directly support that demo is out of scope.

Read first: BRD (FNX-BRD-2026-NO-001), Module 4 (Extraction Engine) and Module 5 (Consistency & Validation). All architectural decisions for the POC trace back to those modules. Use the BRD as ground truth.

SECTION 2

Scope of POC

What we WILL show

- Upload one real submission from Nathan's sample set (recommended: T001732 — has NatHERS + BASIX + colour schedule together)
- Automatic classification of each PDF in the submission (plan set / NatHERS / BASIX)

- Extraction of the windows-and-doors schedule from the NatHERS certificate with all 9 BRD-defined fields (location, W/D number, H×W, type, orientation, glazing, U-value/SHGC, frame, quantity)
- Plan-based extraction of at least 3-5 window tags from the floor plan using Claude vision (proof that vision works, not full coverage)
- BASIX aggregate parse (frame-material totals, glazing totals)
- One demonstrable consistency flag, e.g., dimension mismatch between plans and NatHERS, or BASIX-aggregate mismatch
- Structured output (JSON or simple table) preview of what would feed the final Excel

What we WILL NOT build

- No production UI. A Jupyter notebook or simple Streamlit page is fine
- No user accounts, no authentication, no database, no audit logs
- No deployment, no hosting setup. Runs on a dev laptop during the demo
- No multi-dwelling logic, no revision diff, no rejected queue, no admin dashboard
- No Excel file generation. Show the structured data, explain that XLSX is a known-good downstream step
- No edge-case handling. Pick the cleanest sample and make it shine

SECTION 3

Technical Approach

Stack

- Python 3.11 in a single repo (private GitHub repo under Fonix org, transfer to Nathan's account on kickoff)
- PyMuPDF (fitz) for PDF parsing and page rasterisation
- Anthropic Python SDK, Claude Sonnet 4.6 for vision and text extraction
- Pydantic for the canonical schema (lightweight version, not the full BRD schema)
- Jupyter Notebook OR Streamlit for the demo surface — whichever the dev finds faster
- No frameworks beyond that. Avoid FastAPI, no Docker, no Postgres. Keep it lean

Sample submission to use

- Primary: T001732 set (NatHERS + BASIX + Amended External Colour Schedule). Most complete, three-way reconciliation possible
- Fallback: 11439 Lot 1808 Zillie Close Dubbo (NatHERS cert + stamped plans). Use if T001732 hits an unexpected parsing issue
- Do NOT use 40283 TPA Energy Report — it's aggregate-only and we don't want to demonstrate the hardest path on POC day

Extraction approach per source

Source	Approach	Notes
NatHERS	Text extraction via PyMuPDF + targeted parsing per software (Hero / BERS Pro / FirstRate5)	Primary path. Cleanest data. Start here. Page 5-7 of most NatHERS certs has the itemised schedule
BASIX	Text extraction + table parsing	Pull frame-material and glazing aggregates only. Don't try to itemise
Plans	Rasterise schedule pages at 150 DPI, send to Claude vision with structured prompt	Just prove vision works on a few tags. 3-5 callouts is enough. Don't chase 100% coverage

Reconciliation

- Per BRD M5.1.001: when plans disagree with NatHERS, plans win. For the POC, demonstrate this by flagging at least one mismatch and showing the plan value as the chosen output
- Per BRD M5.4.005: flag types include Dimension mismatch, Orientation mismatch, Window-type mismatch, BASIX-aggregate mismatch. Pick whichever surfaces naturally in the chosen sample

Output

- A Pydantic model dump (JSON) showing the canonical schema populated with extracted data
- A markdown table preview of what the Excel would contain
- A consistency report section listing the flags raised
- Optional: rasterised preview image of a plan page with detected tags highlighted (nice-to-have, defer if it complicates the demo)

SECTION 4

Phases & Roles

Work breakdown in the order it should be tackled. Engineers manage their own pace; the constraint is the demo, not specific phase durations.

#	Phase	Activities
1	Setup	Repo init, environment, Anthropic API key. Open the T001732 PDFs and visually scan them. Identify the NatHERS schedule pages, BASIX aggregate page, and at least one plan sheet with visible window tags
2	Classification + NatHERS	Build the classifier (detect plan vs NatHERS vs BASIX by content). Build the NatHERS schedule parser. Validate against the sample
3	Plans + Vision	Rasterise the chosen plan page, build the Claude vision prompt for tag extraction, validate on 3-5 tags. Highest-risk phase, allocate buffer

#	Phase	Activities
4	BASIX + Reconciliation	BASIX aggregate parse. Build the canonical schema. Wire the three sources into one normalised output
5	Consistency flag	Implement at least one flag type and demonstrate it surfacing on the sample
6	Demo surface	Wrap everything in a Jupyter notebook or Streamlit page. Add comments explaining each step for Nathan to follow along
7	QA + polish	Run end-to-end at least 3 times. Verify the demo flows without errors. Fix any rough edges. Make output look presentable
8	Demo prep + buffer	Lavesh and CTO review the demo. Practice the talk track. Record a Loom backup in case live demo has issues

Role assignments

Role	Person	Responsibility
Lead	Lavesh	Owens the POC end-to-end. Coordinates check-ins. Final demo to Nathan
Technical lead	CTO	Architecture decisions. Reviews the extraction approach. Available for unblocking
Backend / extraction	2 developers	Build classifier, NatHERS parser, Claude vision integration, reconciliation. Pair-program where useful
QA	QA Lead	Run the demo end-to-end before final sign-off. Flag any breakage. Sign off before Lavesh demos

SECTION 5

Acceptance Criteria & Demo Plan

Acceptance criteria — demo passes if ALL of these are true

- Demo runs end-to-end without crashing
- Real Nathan sample used (not synthetic data)
- Classification correctly identifies each PDF in the submission
- NatHERS extraction populates all 9 fields for at least 80% of the windows in the cert
- At least 3 plan-tag extractions work via Claude vision
- BASIX aggregates parse correctly
- At least one consistency flag surfaces
- Final output is structured and readable, suggesting a clear path to the production Excel

Demo flow with Nathan

- Restate what the POC validates: extraction works on your real data
- Show the source PDFs side-by-side (the input)

- Run extraction live, narrate what's happening at each stage
- Show the canonical output and explain how it maps to the BRD's data fields
- Show the consistency flag firing, explain why it matters for Nathan's workflow
- Q&A, set expectations for what gets built in Milestone 2 onwards

What we say if something fails on demo day

Be honest. If the live demo hits an error, switch to the Loom recording recorded during demo prep. Acknowledge the issue briefly, move on. Nathan respects engineers who handle live failures cleanly more than ones who pretend everything is perfect.

Never: Hide failures, blame the data, or promise a fix on the spot without checking with the team.

Risks and contingencies

Risk	Mitigation
Claude vision fails on plan tags	Fall back to demonstrating NatHERS-only extraction. Acknowledge plan extraction needs more iteration in early build phase
Anthropic API rate-limit or outage	Have a recorded Loom backup as fallback (record during demo-prep phase)
T001732 sample has unexpected parsing issue	Switch to Zillie Close fallback sample early. Decision must be made before vision phase starts to avoid wasted effort
Consistency flag implementation incomplete	Demo without consistency flag. Position it as 'this is what we build in Milestone 4'. Still ships the core extraction story